Collections 360: progress report

Team CD Projekt Blaxk · CIBC Collections Hackathon, build phase · status at 16:20 IST, 4 Oct 2026 · deadline 21:00 IST today

In one line: all four layers of the brief now have working code on the real data. What is left is mainly (1) running Layer 2 against the Gemini API once a billing-enabled key works, which also produces the benchmark answers CSV, and (2) the submission items: git commits and push, architecture diagram, demo video and pitch deck.

1. Status by layer

Layer (brief)	Status	Where it stands
1. Data Product Factory	**Done (3 Oct)**	Raw → curated → golden C360 in MongoDB (1,000,008 customer documents). IDs matched across 5 systems with a confidence; 99.95–99.97% of source keys auto-linked, CRM dedup recall 99.85%, 605 mistyped-ID collisions caught. 39 DQ checks, human review queue.
2. Insight and NLP (plain-English questions)	**Code done; live run pending**	Adapted to the real data, policy documents added, evaluation and submission tooling ready. Not yet run against the model: the free Gemini key's quota was used up (20 requests/day).
3. Feature Store	**Done (today)**	One feature definition used for both training and live scoring; 33 structured and text features; batch and live values identical (tested).
4. Models and Decisioning	**Done (today)**	Rules-based next best action with approve/edit/reject (from layer3.zip), now with a trained promise-risk model (out-of-time AUC 0.888), plain-English reasons and a fairness check.

2. Done since Layer 1

2.1 Layer 2 adapted to the real data (runbook Steps 1–6)

- **Code moved to the real data:** the offline suite passed unchanged at first (495 tests). The doctor check then found 20 warnings: guessed column names, and transcript paths relative to `files/`.
- **Schema fixed:** guessed columns were replaced with the real ones in six tables, and transcripts are now found (25,000 of 25,000 parsed). Personal and agent HR fields stay out on purpose. The deny-list was made *stricter*, never looser. The doctor now reports 0 errors and 0 warnings.
- **Serving database:** built from the real data in about 60 s. It now has 25 tables, row counts equal to the curated sources, and c360_id coverage of 98.3–100%.
- **Official metrics:** all 15 official metric definitions were translated and marked verified. 18 metric results were checked against independently hand-written SQL, and **all 18 agree exactly**. Roll rate was sped up from 10 s to 0.1 s with identical results. One finding: monthly snapshots exist only at quarter ends.

- **Benchmark:** 35 questions loaded. Expected answers for the 21 dev questions come from the release labels file; the 14 test answers are hidden. Each question is classified by the route it needs: 14 governed metric, 4 custom SQL, 3 transcript, 9 policy document, 5 refusal.
- **Model:** the Pro models gave HTTP 429 (no quota), and you approved `gemini-3.8-flash`. 4 of 5 live smoke tests passed before its free daily cap of 20 requests ran out. Stopped as instructed.

2.2 layer3.zip merged and connected to the C360 (MongoDB)

- Merged without overwriting anything. Layer 3 includes the agent terminal, next best action (rules, with approve/edit/reject), hardship options, post-call extraction and QA (91–100% accurate on the 500 labels), policy search, and the customer app.
- New `C360_L3_SOURCE=mongo` mode: customer documents now come from Layer 1's golden record in MongoDB, and any key from any system works. Checked end to end on the real data.

2.3 "Block 1": Layer 2 ready to evaluate as soon as the key works

- **Policy documents searchable:** 39 citable sections from 17 current documents, with superseded versions skipped. A new tool, `search_policy`, makes the 9 policy questions answerable.
- **Benchmark answers CSV writer:** follows the release template. It also takes the extra questions released at 19:00 (`--questions-file`), and resumes after a quota stop without repeating calls.
- **Held-out questions:** 18 new real-data questions, each with an answer from hand-written SQL.
- **Performance check:** all 23 metrics together take 1.6 s.
- **Acceptance report generator and a one-command live script:** `scripts/layer2_live.sh`.
- Layer 2 tests: **559 passed**.

2.4 Feature store and promise risk model (brief Layers 3 and 4)

- **One definition:** `c360/features/definitions.yaml`. The same SQL builds the training table (238,815 promises in 2 s) and the live lookup (one promise in 0.1–0.5 s). A test checks 200 random promises every run, and batch and live are identical.
- **Point in time:** each promise only sees data from before its call. A test proves that later data does not change a past promise's features.
- **Features:** structured (promise size versus amount overdue, DPD, contacts, salary gaps, bureau score) and **text**. The text patterns over the call note and transcript were checked against 500 hand-labelled notes: hardship 93% precision and 97% recall. These notes were also used to tune the patterns, so the figures are optimistic.
- **Excluded:** protected attributes, province, language and vulnerability are never used. Voice is left out on purpose: the voice file carries an accent-region field and covers only 2,000 calls.
- **Model:** logistic regression that predicts whether a promise to pay will be broken. Tested out of time on May–Aug 2026, a stress period with a 61% break rate against 37% in training, it scores **AUC 0.888**, with 57.8% predicted against 60.8% actual. Weights must point in the expected direction, so the explanations always make sense.
- **In the agent terminal:** the score is advice inside the existing rules, with exact plain-English reasons. A high-risk promise gets a reminder *call* to check affordability, and hardship, distress and dispute signals appear as safeguards. A person approves every action.
- **Fairness:** checked for measurement only. Results are even across gender, marital status, citizenship,

newcomer status and accessibility. One flag: customers with a recorded vulnerability are placed in the high band 1.27× as often. The report explains why and how this is mitigated.

- Documents: `docs/features/README.md`, the model card, the fairness report and the feature catalogue. Tests: features 11 passed, Layer 3 34 passed.

3. What is left

3.1 Must do before 21:00 IST

#	Task	What is needed	Blocked on
1	**Layer 2 live evaluation**	Run `bash scripts/layer2_live.sh`. It checks the quota first, then runs the smoke tests, the refusal set, the 35 benchmark questions, the held-out questions and the acceptance report. About 100–150 model requests.	**A Gemini key with billing enabled. The current key file is still the free tier (last changed 12:40).**
2	**Extra benchmark questions** (released 19:00)	Save the CSV, then run `bash scripts/layer2_live.sh --only-extra FILE`. This merges everything into `outputs/layer2/benchmark_answers.csv`.	19:00 release + task 1
3	**Submit the benchmark answers CSV**	Submit `outputs/layer2/benchmark_answers.csv`. It is produced automatically; answers are never edited by hand.	Tasks 1–2
4	**Git: commit and push** to the private GitHub repo	Nothing is committed yet, and the brief asks for commits throughout. Commit now in logical steps and push well before 21:00, because commits after the deadline don't count. Data and warehouse folders are already in `.gitignore`.	Repo access
5	**Architecture diagram**	One picture: sources → DuckDB (raw, curated) → MongoDB C360 → Layer 2 serving DB and agent (Gemini) → feature store and model → Layer 3 terminal and app, with the human review points.	—
6	**Pitch deck** (10 slides or fewer)	Problem, design, the four layers with their numbers, safety and fairness, demo screenshots, what's next.	—
7	**Demo video**	Walk through one customer: look up by any key, ask a question in plain English with its SQL shown, a refusal, next best action with the risk score and reasons, approve/edit, post-call QA.	Task 1 for the Layer 2 part
8	**Final README and docs pass**	Add the live Layer 2 results to the README and acceptance report. Optionally add the feature-store tables to the data contract.	Task 1

3.2 Decisions for the team (not blocking the code)

- **"Last N days"** includes the snapshot day (current setting). The benchmark wording supports keeping it.

- **Customer names reach the model:** through `get_customer` and through transcript text in `search_text`. Accept, or mask them?
- **Vulnerability flags in Layer 3:** they are used only to *add* safeguards (certified agent, vulnerable-customer script, never lower priority). Confirm the team will defend this, since vulnerability is listed as a protected attribute.

3.3 Small checks and known issues

- **Promise-risk line not yet seen in a browser:** open one case in the terminal to confirm it shows. The API already returns it, but the JavaScript could not be syntax-checked because Node is not installed.
- **Combined test run:** running Layer 2 and Layer 3 tests in one pytest command gives 2 Layer 3 API failures. These existed before today's work, because Layer 2's test setup switches the process to synthetic data. Each folder passes on its own.
- **Model scores come from Flash:** the Layer 2 scores will reflect `gemini-3.8-flash`, not a Pro model. State this in the README.

3.4 After submission (deferred)

- Block 2: an HTTP API for Layer 2. Block 3: the multi-agent design in `SPEC_LAYER2_MULTIAGENT.md`, which needs Pro quota.
- Voice features, a held-out set for the text patterns, and monthly retraining and drift monitoring for the risk model.

4. Suggested plan for the remaining time

Time (IST)	Work
16:30–17:15	Git: first commits and push. Start the architecture diagram. Get the billing-enabled key working.
17:15–18:30	Layer 2 live run (task 1) in the background. Meanwhile, build the pitch deck.
18:30–19:00	Check the live results and update the README and acceptance report. Commit.
19:00–19:45	Extra questions (task 2), then submit the benchmark answers CSV (task 3). Commit.
19:45–20:40	Record the demo video and finish the deck.
20:40–21:00	Final commit and push, check that every deliverable is in the repo. **No changes after 20:50.**

Sources: CHANGELOG_LAYER2.md, HANDOFF_BLOCK1.md, docs/features/README.md, and the generated reports in docs/ and outputs/.